Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 1.7

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 20/09/2026

```
%load_ext rpy2.ipython
```

```
%%R
mi_vector <- c(1, 2, 3, 4, 5)
mean(mi_vector)
```

```
[1] 3
```

Código en Python (Medición de tiempo)

En esta celda definimos la prueba en Python utilizando time y numpy.

```
import numpy as np
import time

def evaluar_tiempo_python():
    print("--- Ejecutando en Python ---")
    start_time = time.time()

    # Generar matrices aleatorias de 2000x2000
    A = np.random.rand(2000, 2000)
    B = np.random.rand(2000, 2000)

    # Multiplicación de matrices
    C = np.dot(A, B)

    end_time = time.time()
    tiempo_total = end_time - start_time
    print(f"Tiempo de ejecución en Python: {tiempo_total:.4f} segundos\n")
    return tiempo_total

# Ejecutar la función
tiempo_py = evaluar_tiempo_python()
```

```
--- Ejecutando en Python ---
Tiempo de ejecución en Python: 0.8493 segundos
```

Código en R (Medición de tiempo)

Usamos la etiqueta %%R al inicio de la celda. Nota cómo usamos <- para guardar el tiempo en R, pero al final podemos regresar esa variable a Python usando la bandera -o tiempo_r.

```
%%R -o tiempo_r
evaluar_tiempo_r <- function() {
  cat("--- Ejecutando en R ---\n")
  start_time <- Sys.time()

  # Generar matrices aleatorias de 2000x2000
  A <- matrix(runif(2000 * 2000), nrow = 2000, ncol = 2000)
  B <- matrix(runif(2000 * 2000), nrow = 2000, ncol = 2000)

  # Multiplicación de matrices
  C <- A %*% B

  end_time <- Sys.time()
  tiempo_total <- as.numeric(difftime(end_time, start_time, units = "secs"))
  cat(sprintf("Tiempo de ejecución en R: %.4f segundos\n\n", tiempo_total))
  return(tiempo_total)
}

# Ejecutar la función y asignarla a la variable que saldrá a Python
tiempo_r <- evaluar_tiempo_r()
```

```
--- Ejecutando en R ---
Tiempo de ejecución en R: 1.3654 segundos
```

Comparación Final

Como exportamos tiempo_r en la celda anterior, ahora podemos comparar ambos resultados directamente en una celda de Python para ver cuál fue más veloz:

```
print("=== COMPARATIVA DE TIEMPOS ===")
print(f"Python: {tiempo_py:.4f} segundos")
print(f"R:      {tiempo_r[0]:.4f} segundos") # rpy2 devuelve una lista/vector, tomamos el primer elemento

if tiempo_py < tiempo_r[0]:
    ganador = "Python"
    diferencia = tiempo_r[0] / tiempo_py
else:
    ganador = "R"
    diferencia = tiempo_py / tiempo_r[0]

print(f"\n⏳ {ganador} fue el más rápido en esta prueba por un factor de {diferencia:.2f}x.")
```

```
=== COMPARATIVA DE TIEMPOS ===
Python: 0.8493 segundos
R:      1.3654 segundos

⏳ Python fue el más rápido en esta prueba por un factor de 1.61x.
```

Un detalle técnico a tener en cuenta

Al ejecutar esto en Colab, notarás que ambos lenguajes son extremadamente rápidos para esta tarea (suele tomar menos de un segundo). Esto se debe a que por debajo, tanto numpy en Python como las funciones nativas de R en Colab utilizan librerías de álgebra lineal altamente optimizadas (como BLAS/LAPACK) que aprovechan al máximo los hilos del procesador de Google.

Conclusiones finales del estudiante

En esta práctica estuvo interesante ver cómo podemos conectar y hacer convivir Python y R dentro de un mismo cuaderno usando rpy2, sin tener que elegir solo uno. La prueba de rendimiento multiplicando matrices grandes fue muy ilustrativa y entretenida de medir; aunque Python con NumPy resultó ser un poco más rápido, ambos respondieron en poco más de un segundo gracias a las optimizaciones que traen por detrás.

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados